Übungsblatt 5: Der Single-Cycle-Datenpfad

Mikrocomputertechnik 1

Musterlösung

Zielsetzung und Arbeitsweise

Sie erklären die Ausführung von Registerarithmetik, Load, Store und Branch anhand von Datenwegen, Quellenauswahl und Schreibfreigaben. Teil A ist ohne Simulator lösbar und übt typische Klausurformate. Teil B überprüft Ihre Vorhersagen in Ripes.

Teil A: etwa 60 Minuten, 50 Punkte zur Selbsteinschätzung. Teil B: etwa 25 Minuten.

Es gilt das Lehrmodell aus Vorlesung 5:

- Basissubset: `add`, `sub`, `and`, `or`, `lw`, `sw`, `beq`; keine C-Erweiterung.
- 32-Bit-Daten, byteadressierter Speicher, Little Endian; gültige, wortausgerichtete Adressen.
- Register- und Speicherlesen kombinatorisch; Schreiben mit Freigabe an der steigenden Flanke.
- `ALUSrc`: 0 wählt RD2, 1 das Immediate. `MemtoReg`: 0 wählt die ALU, 1 die Speicherlesedaten.
- ImmGen liefert für `beq` bereits den vorzeichenerweiterten Byteoffset einschließlich Bit 0.
- PCSrc = Branch $\wedge$ Zero; 0 wählt $PC + 4$, 1 das Branch-Ziel.

X bezeichnet eine für das Ergebnis unerhebliche Auswahl. Begründen Sie Ihre Antworten; bloße Zahlen reichen bei Erklärungsaufgaben nicht aus.

Teil A: Papieraufgaben

Aufgabe 1: Vom Maschinenwort zum Zustand (10 Punkte)

Bei $PC =$ `0x100` liegt das Maschinenwort `0x00628533` vor. Die Register enthalten `t0 = 50`, `t1 = 100` und `a0 = 7`.

Die R-Type-Felder sind `funct7[31:25]`, `rs2[24:20]`, `rs1[19:15]`, `funct3[14:12]`, `rd[11:7]` und `opcode[6:0]`. Für `add` gelten `opcode = 0x33`, `funct3 = 0` und `funct7 = 0`. Registerzuordnung: `t0 = x5`, `t1 = x6`, `a0 = x10`.

1. Decodieren Sie `rs1`, `rs2` und `rd` und notieren Sie die Assemblerinstruktion. (3 P.)
2. Geben Sie A1, A2, A3 sowie RD1, RD2 und WD vor der nächsten Flanke an. Unterscheiden Sie Nummern und Werte. (3 P.)
3. Nennen Sie die Änderungen an der Flanke und erklären Sie, warum `a0` unmittelbar davor noch 7 enthält. (2 P.)
4. Unabhängiger Fall mit denselben Anfangswerten: Ersetzen Sie den Befehl durch `add t0, t0, t1`. Warum entsteht keine endlose Rückkopplung? (2 P.)

Musterlösung Aufgabe 1

1. `rs1 = 5`, `rs2 = 6`, `rd = 10`; die Funktionsfelder ergeben `add a0, t0, t1`.
2. Adressports: `A1 = 5`, `A2 = 6`, `A3 = 10`, jeweils fünf Bit. Datenports: `RD1 = 50 = 0x00000032`, `RD2 = 100 = 0x00000064`, `WD = 150 = 0x00000096`, jeweils 32 Bit.
3. Mit `RegWrite = 1` übernimmt `a0` den Wert 150, der PC wird `0x104`. Quellregister und Datenspeicher bleiben unverändert. Vor der Flanke ist 150 nur vorbereitet; die Registerbank speichert weiterhin den alten Zustand.
4. Der Leseport liefert zunächst den alten Wert 50. Die ALU berechnet 150, das erst an der Flanke in `t0` übernommen wird. Der flankengetriggerte Registerzustand unterbricht die kombinatorische Rückführung; anschließend wird die nächste Instruktion betrachtet.

Aufgabe 2: Load und Store mit negativem Offset (12 Punkte)

Die folgenden zwei Instruktionen werden nacheinander ausgeführt:

```
# erster Befehl bei PC = 0x180
lw a0, -4(s1)
sw a0,  4(s1)
```

Anfangszustand: `s1 = 0x1004`, `a0 = 99`. Speicherwörter: `0x1000: 88`, `0x1004: 99`, `0x1008: 0x12345678`.

1. Geben Sie die zwölf Offsetbits und das 32-Bit-ImmGen-Ergebnis für den Load an. (2 P.)
2. Ergänzen Sie für beide Befehle: effektive Adresse, ALU-Operation, Quelle des zweiten ALU-Operanden und Schreibziel. (4 P.)
3. Welcher Wert steht nach dem Load in `a0`? Welche Werte stehen nach dem Store in `a0` und den drei Speicherwörtern? (3 P.)
4. Notieren Sie die vier Bytewerte ab `0x1008` nach dem Store und den PC nach beiden Befehlen. (2 P.)
5. Warum gelangt beim Store der Inhalt von `a0` zum Speicher, obwohl `ALUSrc = 1` ist? (1 P.)

Musterlösung Aufgabe 2

1. Zwölf Bits: `1111 1111 1100`; vorzeichenerweitert: `0xFFFFFFFC`, also -4.
2. Load: `0x1004`$-4 =$ `0x1000`, ALU = ADD, Quelle B = Immediate, Schreibziel = Register `a0` mit Speicherlesedaten. Store: `0x1004`$+ 4 =$ `0x1008`, ALU = ADD, Quelle B = Immediate, Schreibziel = Speicherwort mit Daten aus RD2.
3. Nach `lw` ist `a0 = 88`. Nach `sw` bleibt `a0 = 88`. Die Wörter sind dann `0x1000: 88`, `0x1004: 99`, `0x1008: 88`.
4. In aufsteigender Adressreihenfolge: `58 00 00 00` (Hex, Little Endian). Der PC ist nach dem Load `0x184`, nach dem Store `0x188`.
5. RD2 speist einen eigenen Speicher-Schreibdatenweg. ALUSrc wählt nur den zweiten ALU-Eingang zur Adressberechnung und trennt diesen Abzweig nicht ab.

Aufgabe 3: Branch-Entscheidung und Byteoffset (12 Punkte)

Ein `beq t0, t1, ziel` liegt bei `PC = 0x200`, das Label `ziel` bei `0x20C`. Betrachten Sie zwei unabhängige Fälle: A mit `t0 = 5`, `t1 = 5`; B mit `t0 = 5`, `t1 = 6`.

1. Bestimmen Sie den Byteoffset und beide PC-Kandidaten. (3 P.)
2. Ergänzen Sie die Tabelle. Geben Sie die ALU-Differenz als 32-Bit-Hexwort an. (4 P.)

Fall	ALU-Differenz	Branch	Zero	PCSrc	PC nach der Flanke
A					
B					

3. Ein anderer Branch bei derselben PC-Adresse hat den Offset −8. Bestimmen Sie ImmGen-Ausgang und Zieladresse. (2 P.)
4. Warum wird der ImmGen-Ausgang vor dem Branch-Addierer nicht noch einmal nach links geschoben? (1 P.)
5. Unabhängiger Fall: `sub a0, t0, t1` mit gleichen Operanden bei `PC = 0x200`. Bestimmen Sie Zero, PCSrc und den neuen PC und begründen Sie den Unterschied zu Fall A. (2 P.)

Musterlösung Aufgabe 3

1. Der Offset ist +12 Byte: `0x20C` − `0x200`. Kandidaten sind `0x204` und `0x20C`, beide aus dem alten PC berechnet.

Fall	ALU-Differenz	Branch	Zero	PCSrc	PC nach der Flanke
A	`0x00000000`	1	1	1	`0x20C`
B	`0xFFFFFFFF`	1	0	0	`0x204`

2. Die Tabelle zeigt: Vergleich und Branch-Freigabe müssen beide wahr sein.
3. Offset −8: ImmGen = `0xFFFFFFF8`, Ziel = `0x1F8`.
4. ImmGen hat das implizite Bit 0 bereits eingefügt und liefert den fertigen Byteoffset. Ein weiterer Shift würde die Distanz verdoppeln; im ersten Beispiel entstünde irrtümlich `0x218`.
5. Bei `sub` ist `Zero = 1`, aber `Branch = 0`. Damit PCSrc = $0 \wedge 1 = 0$, neuer PC = `0x204`. `a0` erhält null. Zero allein löst keinen Branch aus.

Aufgabe 4: Steuersignale und Fehlersuche (8 Punkte)

1. Ergänzen Sie die Tabelle für unser Lehrmodell. Verwenden Sie X, wenn eine Auswahl das Ergebnis nicht beeinflusst, und benennen Sie die ALU-Funktion. (4 P.)

Befehl	ALUSrc	MemtoReg	RegWrite	MemRead	MemWrite	Branch	ALU
`add`							
`lw`							
`sw`							
`beq`							

2. Bei `lw a0, 4(s1)` seien `s1 = 0x1000` und das Wort an `0x1004` gleich 88. Nur `MemtoReg` steht fälschlich auf 0. Welcher Wert wird in `a0` geschrieben? (2 P.)
3. Bei einem korrekten `sw` wird nur `MemtoReg` von 0 auf 1 geändert. Ändert das den Speicher-Schreibwert? Begründen Sie die Antwort über Datenweg und Freigabe. (2 P.)

Musterlösung Aufgabe 4

Befehl	ALUSrc	MemtoReg	RegWrite	MemRead	MemWrite	Branch	ALU
`add`	0	0	1	0	0	0	ADD
`lw`	1	1	1	1	0	0	ADD
`sw`	1	X	0	0	1	0	ADD
`beq`	0	X	0	0	0	1	SUB

2. `a0` erhält die ALU-Adresse `0x00001004` statt des Speicherworts `0x00000058`.
3. Nein. MemtoReg wirkt nur am Register-Rückschreibport, dessen Freigabe beim Store null ist. Der Speicher-Schreibwert kommt direkt aus RD2. X bezeichnet eine unerhebliche Auswahl, keinen unbekannten Leitungspegel.

Aufgabe 5: Laufzeiten und gemeinsamer Takt (8 Punkte)

Erfundene Lehrwerte, keine Messung:

Bestandteil	Verzögerung
Clock-to-Q	0,2 ns
Befehlsspeicher	2,0 ns
Registerlesen	1,0 ns
ALU	1,5 ns
Datenspeicherlesen	1,8 ns
Write-Back-MUX	0,5 ns
Setup	0,3 ns

Annahmen: Operanden- und Steuerpfade kommen nicht später an. Alle übrigen benötigten Wege einschließlich Store und PC-Update sind nicht länger als der Load-Weg. Clock-Skew und Reserve werden vernachlässigt.

1. Geben Sie den `add`- und den `lw`-Rückschreibweg an und berechnen Sie jeweils die erforderliche Zeit. (4 P.)
2. Bestimmen Sie die kleinste gemeinsame Taktperiode und die ideale maximale Frequenz. (2 P.)
3. Welche Taktperiode erhält ein `add` in diesem Entwurf? Bei einem `beq` laufen Zielbildung und Vergleich parallel: Werden ihre Verzögerungen addiert? Begründen Sie beide Antworten. (2 P.)

Musterlösung Aufgabe 5

1. `add`: PC → Befehlsspeicher → Registerlesen → ALU → Write-Back-MUX → Zielregister.

$$T_{\text{add}} = 0{,}2 + 2{,}0 + 1{,}0 + 1{,}5 + 0{,}5 + 0{,}3 = 5{,}5\,\text{ns}$$

   `lw`: derselbe Weg, zusätzlich Datenspeicherlesen zwischen ALU und Write-Back-MUX.

$$T_{\text{lw}} = 5{,}5 + 1{,}8 = 7{,}3\,\text{ns}$$

2. Unter den gegebenen Annahmen: $T_{\text{clk,min}} = 7{,}3\,\text{ns}$; $f_{\text{max}} = 1/T_{\text{clk,min}} \approx 137\,\text{MHz}$.
3. Auch `add` erhält dieselbe gemeinsame Periode von mindestens 7,3 ns. Für parallele Branch-Zweige ist die spätere Ankunft maßgeblich; die Zeiten werden nicht einfach addiert. Nach der Zusammenführung ist die nachfolgende Auswahl- und Setup-Verzögerung noch zu berücksichtigen.

Teil B: Ripes-Traces

Aufgabe 6: Vorhersage und Beobachtung vergleichen

Wählen Sie einen RV32-Single-Cycle-Prozessor und deaktivieren Sie C. Notieren Sie Ripes-Version und Prozessorbezeichnung. Nutzen Sie, soweit vorhanden, das erweiterte Layout und die Tooltips.

Die folgenden Dateien liegen unter `beispiele/`. Ihre `li`-/`la`-Anweisungen sind Setup und gehören nicht zum Basissubset. Prüfen Sie im Disassembly, welche echten Instruktionen daraus entstehen. Verwenden Sie keine feste Schrittzahl pro Quelltextzeile und keine fest vermutete Segmentbasis.

Halten Sie jeweils an, wenn der PC auf das untersuchte Label zeigt. Notieren Sie die Werte **vor Clock**, führen Sie genau einen Schritt aus und prüfen Sie den Zustand **danach**. Ein Ripes-Exit ist für diese Beobachtung nicht nötig; halten Sie vor den Exit-Anweisungen an.

A: Registerarithmetik

Laden Sie `mct1_vl5_rtype_trace.s` und halten Sie bei `trace_add` an.

- Welche Registerwerte und welcher ALU-Schreibwert liegen vor?
- Welche Quelle wird zum Registerschreibport gewählt, und ist seine Freigabe aktiv?
- Was ändert sich an `a0` und PC nach Clock?

B: Load und Store

Laden Sie `mct1_vl5_load_store_trace.s`. Lesen Sie die tatsächliche Basisadresse B von `werte` ab.

- Bei `trace_lw`: Offset, effektive Adresse, Speicherlesewert und Registerschreibquelle notieren. Danach `a0` prüfen.
- Bei `trace_sw`: Adresse, Speicher-Schreibdaten und Freigaben notieren. Danach beide Wörter und die vier Byte des ersten Worts prüfen.
- Ordnen Sie beobachtete Funktionen den Lehrmodell-Signalen zu, ohne ungeprüft dieselben MUX-Portnummern vorauszusetzen.

C: Branch genommen und nicht genommen

Laden Sie `mct1_vl5_branch_trace.s`. Durchlauf A nutzt `t1 = 5`, Durchlauf B `t1 = 6` bei unverändertem `t0 = 5`. Initialisieren Sie vor jedem Durchlauf neu.

- Bestimmen Sie die Adressen von `trace_beq`, `not_taken` und `taken` im Disassembly.
- Vergleichen Sie Zieloffset, Branch-Entscheidung und PC nach Clock.
- Prüfen Sie `a0` bei `done`. Welches Ripes-Bauteil führt Vergleich und Zielrechnung aus?

Trace	PC und Befehl vor Clock	Operanden / Quellen / Freigaben	Vorhergesagt danach	Beobachtet danach

Trace	PC und Befehl vor Clock	Operanden / Quellen / Freigaben	Vorhergesagt danach	Beobachtet danach
A: add				
B: lw				
B: sw				
C: beq, gleich				
C: beq, ungleich				

Musterlösung Aufgabe 6: funktionale Erwartungen

- A: `RD1 = 50 = 0x32`, `RD2 = 100 = 0x64`, ALU- und Registerschreibwert `150 = 0x96`; Registerfreigabe aktiv. Danach `a0 = 150`, PC bei `after_add`.
- B, Load: Offset 4, Adresse $B + 4$, gelesener Wert `88 = 0x58`, Rückschreibquelle Speicher. Danach `a0 = 88`.
- B, Store: Offset 0, Adresse B, Daten 88 aus `a0`. Speicherfreigabe aktiv, Registerfreigabe inaktiv. Danach stehen an B und $B + 4$ jeweils 88; die Bytefolge am ersten Wort ist `58 00 00 00`.
- C: Zieloffset +12 Byte, ohne zweiten Shift. Bei Gleichheit geht der PC zu `taken` und `a0` wird später 10. Bei Ungleichheit geht er zu `not_taken` und `a0` wird später `0xFFFFFFFF`.
- Portnamen, MUX-Codierungen sowie Vergleichs- und Zielrechenbauteile sind anhand der verwendeten Ripes-Version zu dokumentieren.